

**23L601 – DIGITAL SIGNAL PROCESSING
REAL-TIME AUDIO LOOPBACK SYSTEM**

SUBMITTED BY:

ATHIKA MA -- 23L211

SATHISH M --23L261

AKILESHWARAN P -- 24L431

ASWATH P – 24L432

BACHELOR OF ENGINEERING

BRANCH: ELECTRONICS AND COMMUNICATION ENGINEERING Of

Anna University



MARCH 2026

COURSE CODE:23L601

COURSE: DIGITAL SIGNAL PROCESSING

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

PSG COLLEGE OF TECHNOLOGY

(AUTONOMOUS INSTITUTION)

COIMBATORE-641004

ACKNOWLEDGEMENT

We extend our heartfelt gratitude to our Principal, **Dr. K. Prakasan**, for granting us the opportunity and providing the essential resources and infrastructure to successfully carry out this project work.

We are deeply thankful to **Dr. V. Krishnaveni**, Professor and Head, Department of Electronics and Communication Engineering, for her constant support, motivation, and encouragement throughout the duration of the project.

We express our sincere appreciation to our Programme Coordinator, **Dr P. Kalpana**, Associate Professor, Department of ECE, and our Tutor, **Dr C. Ramya**, Assistant Professor, Department of ECE, for their continuous guidance, valuable insights, and thoughtful feedback at every stage of our work.

Our heartfelt thanks go to **Dr C. Ramya**, Assistant Professor, Department of ECE, for their mentorship and unwavering encouragement, which played a pivotal role in the successful completion of our project.

Lastly, we would like to acknowledge and thank our senior students and colleagues from the department, whose timely help and cooperation contributed immensely to the smooth and hassle-free execution of this project.

ABSTRACT

This project presents the design and implementation of a real-time audio signal acquisition and processing system using the TMS320C6748 DSP processor. The system captures analog audio signals through a microphone, converts them into digital form, and processes them in real time using the DSP architecture. The processed signal is then transmitted to an output device such as a speaker with minimal delay, ensuring low-latency performance.

The implementation utilizes peripherals such as McASP for audio data transfer and I2C for codec configuration, enabling efficient communication between the DSP and the audio codec. The system demonstrates reliable real-time audio handling and highlights the advantages of dedicated DSP processors in signal processing applications.

This project can be further extended to advanced applications such as noise reduction, echo cancellation, speech enhancement, and real-time communication systems.

INTRODUCTION:

Digital Signal Processing (DSP) plays a crucial role in modern electronic systems, enabling efficient processing of real-time signals such as audio, speech, and images. Unlike general-purpose processors, DSP processors are specifically designed to handle continuous data streams with high speed and low latency, making them ideal for real-time applications.

Audio signal processing is one of the most important applications of DSP, widely used in communication systems, multimedia devices, and embedded systems. It involves acquiring analog audio signals, converting them into digital form, processing them, and reproducing them with minimal distortion and delay.

In this project, the TMS320C6748 DSP processor is used to implement a real-time audio signal acquisition and processing system. The system captures audio input through a microphone, processes it using DSP techniques, and outputs the signal through a speaker. By utilizing peripherals such as McASP for audio data transfer and I2C for codec configuration, the system ensures efficient communication and synchronization.

This project demonstrates the capability of DSP processors in handling real-time audio applications and provides a foundation for developing advanced audio processing systems such as noise reduction, speech enhancement, and communication interfaces.

PROBLEM STATEMENT:

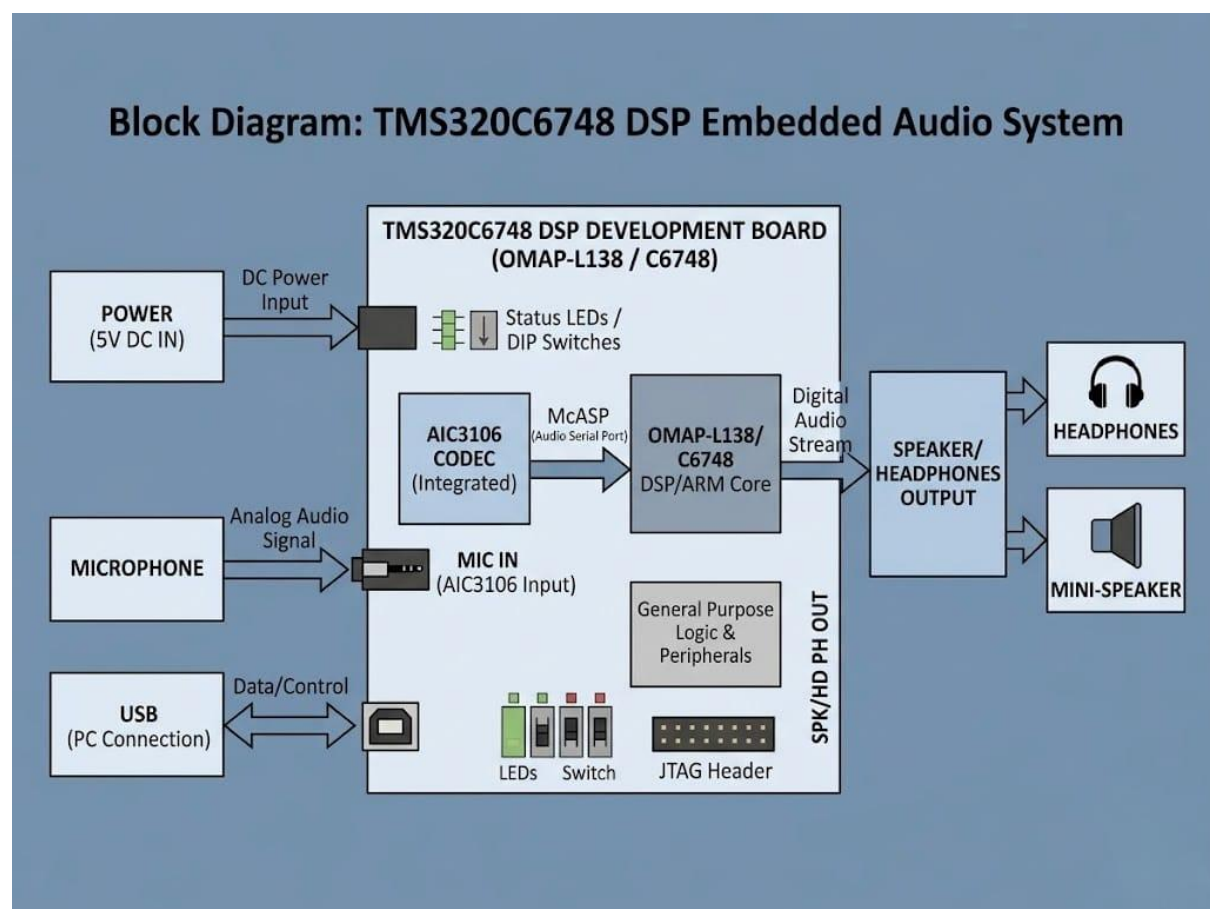
Conventional audio processing systems implemented on general-purpose processors often face challenges such as higher latency and inefficient real-time performance. These limitations affect applications that require immediate audio response, such as live communication and signal monitoring.

Therefore, there is a need for a dedicated DSP-based system that can efficiently acquire, process, and reproduce audio signals in real time with minimal delay and high reliability.

OBJECTIVES:

- To design a system for real-time audio signal acquisition using a microphone
- To implement audio signal processing using the TMS320C6748 DSP
- To achieve low-latency audio transmission from input to output
- To interface audio codec using I2C protocol
- To enable real-time data transfer using McASP peripheral

BLOCK DIAGRAM:



System Description

The system is built using the TMS320C6748 DSP along with the AIC3106 audio codec. The codec is responsible for analog-to-digital and digital-to-analog conversion, while the DSP performs real-time processing. Communication between DSP and codec is achieved using McASP and I2C protocols.

METHODOLOGY

The system begins by capturing the user's voice through a microphone. Since the microphone produces an analog electrical signal, the captured audio cannot be processed directly by the DSP. Therefore, the analog signal is first sent to the audio codec, where the Analog-to-Digital Converter (ADC) converts it into a digital signal.

After conversion, the digital audio samples are transferred from the codec to the TMS320C6748 DSP through the McASP (Multichannel Audio Serial Port) interface. The McASP peripheral is responsible for high-speed and synchronized transfer of audio data between the codec and the DSP. At the same time, the I2C communication interface is used to configure the codec registers such as sampling rate, gain, input/output selection, and other audio parameters.

Once the audio samples reach the DSP, the processor continuously reads the incoming data in real time. In this project, the DSP immediately sends the received signal back to the output path after processing. The processing may include operations such as amplification, filtering, or volume adjustment depending on the implemented algorithm. In the given implementation, the received audio data is multiplied by a constant value before being transmitted back, thereby increasing the volume of the signal.

The processed digital signal is then transmitted back to the audio codec through the McASP interface. Inside the codec, the Digital-to-Analog Converter (DAC) converts the processed digital samples back into an analog audio signal. This analog output is finally delivered to a speaker or headphones, allowing the user to hear the processed version of the original voice.

The entire sequence of acquisition, conversion, processing, and output occurs continuously in a loop with extremely small delay. Because the TMS320C6748 DSP is specifically designed for high-speed signal processing applications, the system is able to achieve real-time performance with very low latency, making it suitable for applications such as voice communication, audio monitoring, and live sound processing.

Hardware Requirements

- TMS320C6748 DSP Development Board
- Audio Codec (AIC3106)
- Microphone
- Speaker / Headphones
- Power Supply

Software Requirements

- Code Composer Studio (CCS)
- Embedded C Programming
- DSP Libraries

APPENDIX

SOURCE CODE

1. Header Files and Definitions

```
#include "VSK_6748.h" // Board support library for TMS320C6748

#ifndef NULL
#define NULL 0
#endif

#define I2C_PORT_AIC3106 (I2C0) // I2C port used for codec communication
#define I2C_TIMEOUT (500000) // Timeout for I2C operations
```

2. Global Variables

```
static i2c_clk_e g_clock_rate; // Stores I2C clock configuration
```

3. McASP Initialization

```
uint32_t MCASP_init(void)
{
    // Enable McASP peripheral
    EVMC6748_lpscTransition(PSC1, DOMAIN0, LPSC_MCASP0, PSC_ENABLE);

    MCASP->GBLCTL = 0; // Reset McASP

    // Configure receive section
    MCASP->RMASK = 0xFFFFFFFF; // Receive mask
    MCASP->RFMT = 0x00008078; // Receive format

    // Configure transmit section
    MCASP->XMASK = 0xFFFFFFFF; // Transmit mask
    MCASP->XFMT = 0x00008078; // Transmit format
```

```

// Configure serializers (3 = transmit, 4 = receive)
MCASP->SRCTL3 = 0x000D; // Transmitter
MCASP->SRCTL4 = 0x000E; // Receiver

// Set pin directions
MCASP->PDIR = 0x14000008;

return (ERR_NO_ERROR);
}

```

4. Audio Codec Initialization (AIC3106)

```

uint32_t AIC3106_init(void)
{
    // Select register page and reset codec
    AIC3106_writeRegister(AIC3106_REG_PAGESELECT, 0);
    AIC3106_writeRegister(AIC3106_REG_RESET, 0x80);

    // Configure codec registers (sampling, routing, gain)
    AIC3106_writeRegister(3, 0x22);
    AIC3106_writeRegister(4, 0x20);

    // Enable headphone outputs
    AIC3106_writeRegister(51, 0x09);
    AIC3106_writeRegister(65, 0x09);

    return (ERR_NO_ERROR);
}

```


5. I2C Initialization

```
uint32_t I2C_init(i2c_regs_t *i2c, i2c_clk_e in_clock_rate)
{
    i2c->ICMDR = 0;    // Reset I2C module

    i2c->ICPSC = 2;    // Set prescaler

    // Set clock for fast mode (400 kHz)
    i2c->ICCLKL = 5;
    i2c->ICCLKH = 5;

    SETBIT(i2c->ICMDR, IRS); // Enable I2C

    return (ERR_NO_ERROR);
}
```

6. Audio Interface Start

```
uint32_t testAudioLineIn(void)
{
    // Enable high-frequency clocks
    SETBIT(MCASP->XGBLCTL, XHCLKRST);
    SETBIT(MCASP->RGBLCTL, RHCLKRST);

    // Enable bit clocks
    SETBIT(MCASP->XGBLCTL, XCLKRST);
    SETBIT(MCASP->RGBLCTL, RCLKRST);

    // Clear serializers
    SETBIT(MCASP->XGBLCTL, XSRCLR);
    SETBIT(MCASP->RGBLCTL, RSRCLR);
}
```

```

MCASP->XBUF3 = 0; // Avoid underrun

// Enable state machines
SETBIT(MCASP->XGBLCTL, XSMRST);
SETBIT(MCASP->RGBLCTL, RSMRST);

// Start frame sync
SETBIT(MCASP->XGBLCTL, XFRST);
SETBIT(MCASP->RGBLCTL, RFRST);

return (ERR_NO_ERROR);
}

```

7. Main Function

```

int main(void)
{
    int32_t dat; // Variable to store audio sample

    USTIMER_init(); // Initialize timer

    I2C_init(I2C0, I2C_CLK_400K); // Initialize I2C

    MCASP_init(); // Initialize McASP

    AIC3106_init(); // Initialize audio codec

    testAudioLineIn(); // Start audio interface

    // Infinite loop for real-time processing
    while(1)

```

```

{
    // Wait until transmit buffer is ready
    while (!CHKBIT(MCASP->SRCTL3, XRDY)) {}

    dat = MCASP->XBUF4; // Read input audio sample

    MCASP->XBUF3 = dat * 9; // Amplify and send to output
}
}

```

Actual Dsp Processing (Amplification)

```
MCASP->XBUF3 = dat * 9;
```

Software Implementation:

The system begins by initializing the timer, I2C interface, McASP module, and the audio codec. The codec is configured using I2C communication, while McASP is used for real-time audio data transfer.

After initialization, the program enters an infinite loop where it continuously checks for incoming audio data. Once data is available, it is read from the receive buffer and processed. In this implementation, the audio signal is amplified by multiplying it with a constant factor.

The processed signal is then written back to the transmit buffer, enabling real-time audio output through the speaker.

RESULTS

The implemented system successfully captures and processes audio signals in real time with minimal delay. The output audio closely follows the input signal, demonstrating efficient data acquisition and processing. The system confirms the capability of the TMS320C6748 DSP in handling real-time audio applications effectively.



Advantages of the Proposed System

- **Real-Time Processing:**

The system processes audio signals instantly with minimal delay, making it suitable for live audio applications.

- **Low Latency Performance:**

Due to the use of the TMS320C6748 DSP, the delay between input and output is extremely small, ensuring smooth audio reproduction.

- **Efficient DSP Architecture:**

The dedicated DSP processor is optimized for signal processing tasks, providing high computational efficiency compared to general-purpose processors.

- **High-Speed Data Transfer:**

The use of McASP enables fast and synchronized audio data communication between the DSP and the audio codec.

- **Flexible and Scalable Design:**

The system can be easily extended to include advanced features such as filtering, noise reduction, and speech processing.

- **Hardware-Level Control:**

The use of I2C communication allows precise control over codec parameters like gain, sampling rate, and routing.

Future Scope

- Implementation of noise reduction techniques
- Integration of echo cancellation algorithms
- Real-time speech recognition
- Wireless audio transmission
- Audio effects and filtering applications

Conclusion

The project successfully demonstrates the design and implementation of a real-time audio signal acquisition and processing system using the TMS320C6748 DSP. The system achieves low-latency performance and reliable operation, making it suitable for various real-time audio applications. This highlights the effectiveness of DSP processors in handling continuous signal processing tasks.